

Polynomial Regression: Assignment 1

Roll no. BT2024171 | Phase 1: turbine power score (var1) | Phase 2: thermal reservoir map (var2)

d = 5

var1 chosen degree

R² = 0.958

var1 5-fold CV (MSE 0.416)

d = 12

var2 chosen degree

R² = 0.995

var2 5-fold CV (MSE 0.235)

1. Problem and data

The task is to predict a continuous target y from the inputs using only polynomial regression, and to choose the polynomial degree carefully so the model neither underfits nor overfits. Two personalised datasets are used (the files matching roll number BT2024171). A polynomial of degree d contains every term whose powers of the features add up to at most d .

	var1 (turbine)	var2 (reservoir)
Features	6 ($x_1 \dots x_6$)	3 ($x_1 \dots x_3$)
Max degree allowed	10	20
Train / test rows	1000 / 1000	1000 / 1000
Missing values	0	0
Target y range (train)	-10.6 to 14.8	-28.3 to 40.0
Target mean / std (train)	0.76 / 3.17	2.29 / 6.85
Terms at max degree	8007	1770

2. Method

Each input vector x is expanded into all monomials of total degree at most d , $\phi(x) = [x_1, x_2, \dots, x_1^2, x_1 x_2, \dots, x_k^d]$, with $k = 6$ (var1) or $k = 3$ (var2). The model is $\hat{y} = b + w \cdot \phi(x)$. The number of terms is $C(k+d, d) - 1$, which grows very fast: 461 terms for var1 at $d = 5$ but 8007 at $d = 10$; 454 for var2 at $d = 12$ but 1770 at $d = 20$. Training has only 1000 rows, so high degrees have more parameters than data points, and because the inputs lie in $[-1, 1]$ the high powers are nearly collinear.

To keep the fit stable we minimise the least-squares error plus an L2 (ridge) penalty on the weights (the intercept b is not penalised):

$$\min_{b, w} \sum_{i=1}^n (y_i - b - w^T \phi(x_i))^2 + \alpha \|w\|_2^2$$

With $\alpha \rightarrow 0$ this is ordinary polynomial least squares; larger α shrinks the coefficients and reduces variance. The model is still a polynomial in the inputs. Nothing else (no other model family, no feature scaling, no extra preprocessing) was used, because the data are already clean.

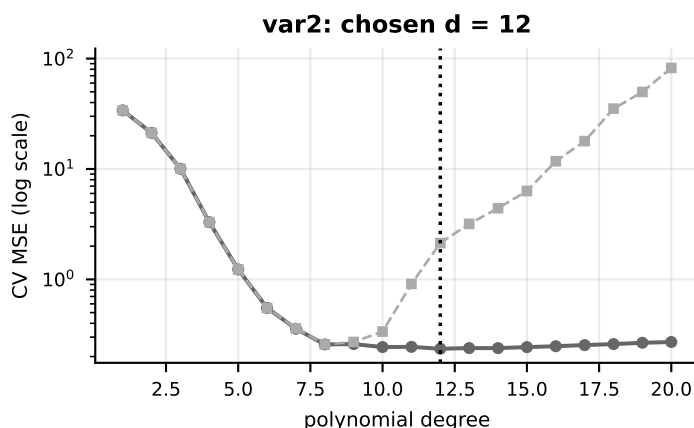
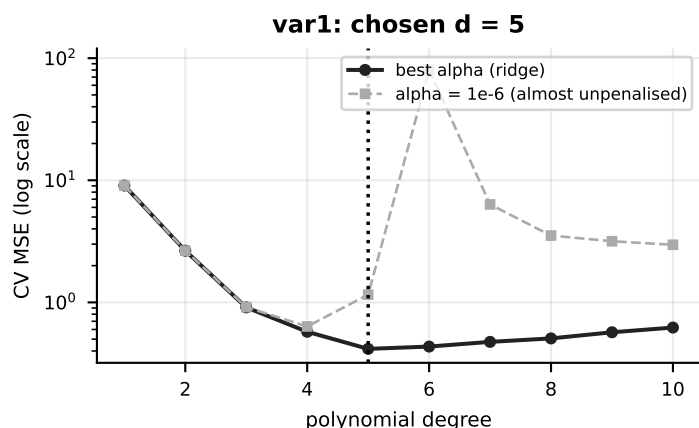
3. Model selection protocol

For every degree d (1..10 for var1, 1..20 for var2) and every α in $\{1e-6, 1e-4, 1e-2, 0.1, 1, 3.16, 10, 100\}$ we measured the mean squared error of 5-fold cross-validation (shuffled, fixed seed) on the training set only. The (d, α) pair with the lowest CV MSE was taken; when several degrees were statistically tied, the lowest was preferred (simpler model, safer to extrapolate). A finer α grid with 3x repeated CV was used to confirm the close calls. The test files were never used to fit or tune anything.

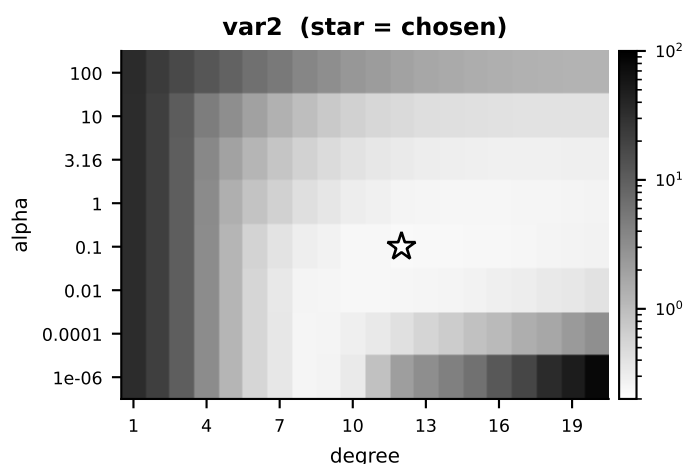
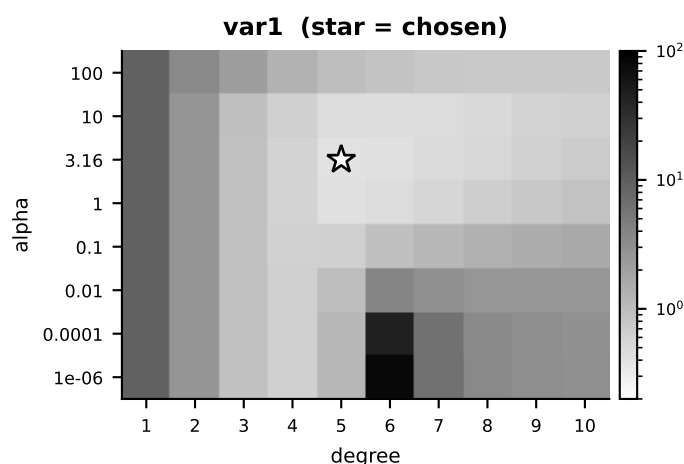
Choosing the degree and the penalty

5-fold cross-validated MSE on the training data

4. Degree selection curves (best alpha at each degree)



5. Joint search over degree and alpha (CV MSE, log colour scale)



6. What the curves show

var1 (up to degree 10). CV MSE falls from 9.05 at $d = 1$ to 0.416 at $d = 5$: below 5 the polynomial is too simple (underfitting; the error at $d = 3$ is still 0.91). After $d = 5$ the error rises again (0.435 at $d = 6$, 0.622 at $d = 10$) because the 8000-term model starts to fit noise (overfitting). $d = 5$ and $d = 6$ are tied in repeated CV (0.433 vs 0.436), so $d = 5$ was chosen. The best alpha is moderate (about 3) and grows with degree (about 10 for $d \geq 7$): the richer the model, the stronger the penalty it needs.

var2 (up to degree 20). The error drops steeply until $d = 8$ (0.257) and then stays on a plateau of about 0.23 to 0.27 up to $d = 20$. Repeated CV with a finer alpha grid gave 0.2375 ($d = 10$), 0.2290 ($d = 12$) and 0.2295 ($d = 14$): statistically equivalent, so the lowest, $d = 12$, was selected. The curve is flat rather than U-shaped only because of the ridge penalty; the grey dashed line shows what happens without it (CV MSE 12 at $d = 16$ for var2 and 3.0 at $d = 10$ for var1).

Why not simply the highest degree? Once the plateau is reached, a higher degree is not rewarded; it only adds parameters and extrapolates worse (see Section 10). The lowest adequate degree is the more defensible choice.

Results

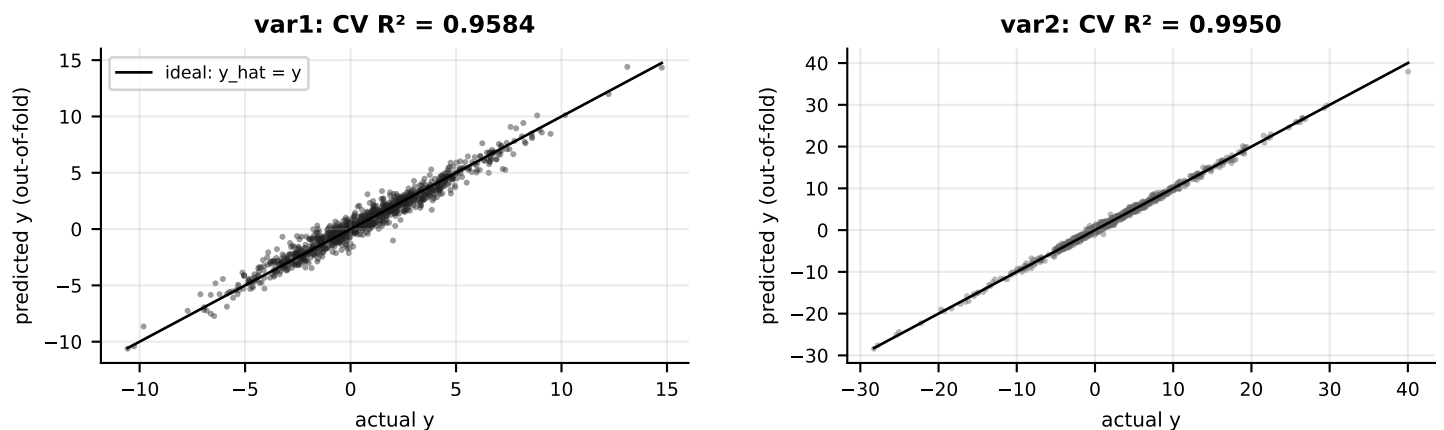
Cross-validated performance of the final models and fit quality

7. Final models

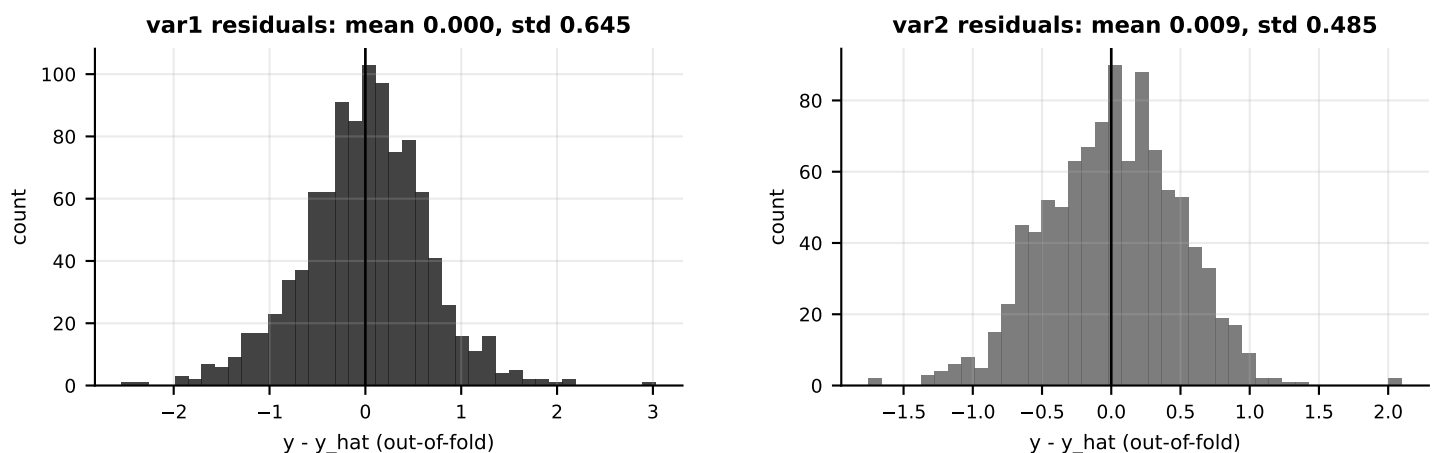
Metric	var1	var2
Polynomial degree	5	12
Ridge alpha	3.16	0.10
Number of terms (excl. intercept)	461	454
Train MSE / R ²	0.1866 / 0.9814	0.1486 / 0.9968
5-fold CV MSE / R ²	0.4161 / 0.9584	0.2351 / 0.9950

MSE = $\text{mean}((y - y_{\text{hat}})^2)$ and $R^2 = 1 - \text{sum}((y - y_{\text{hat}})^2) / \text{sum}((y - \text{mean}(y))^2)$. Train metrics come from fitting on all 1000 rows; CV metrics use out-of-fold predictions, so they estimate performance on unseen points from the same distribution. The gap between train and CV MSE (0.19 vs 0.42 and 0.15 vs 0.24) is the expected optimism of in-sample fitting and is small, so the regularised models are not badly overfit.

8. Out-of-fold predictions vs. actual values



9. Residuals and test-set predictions



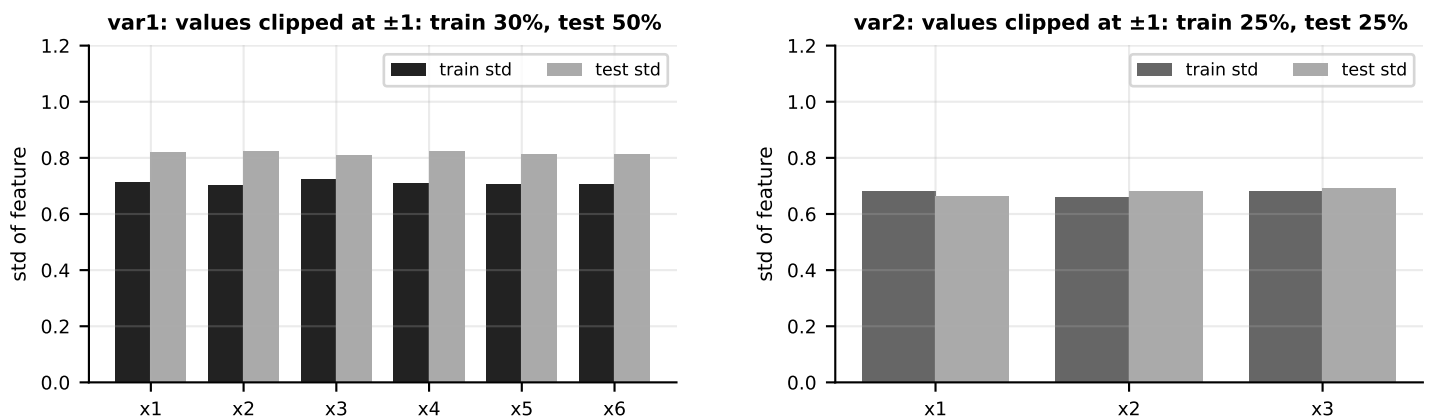
The scatter plots follow the diagonal over the whole target range, and the residuals are centred near zero with no obvious structure, which suggests the polynomial family captures the underlying surface and the remaining error behaves like noise. The residual std (0.65 for var1, 0.48 for var2) is the error level to expect.

Test predictions (1000 rows each) were produced by refitting each model on all 1000 training rows. Their ranges are var1: -10.5 to 15.1 (train y: -10.6 to 14.8) and var2: -29.5 to 37.6 (train y: -28.3 to 40.0). They are in the same range as the training targets; no wild extrapolation values appear.

Discussion, limitations and reproducibility

Distribution shift between train and test inputs, and how the code is organised

10. Train vs. test input distribution



All features lie in $[-1, 1]$ and a sizeable share of values sits exactly on the boundary (the data are clipped). For var1 the test features are clearly more spread out than the training features (std about 0.82 against 0.71) and have more boundary values (50% against 30%), so many test points sit nearer the edges and corners of the input cube, where there are fewer training samples. For var2 the shift is small. This matters because polynomials extrapolate badly: a high-degree term such as x^{10} grows steeply near the edge, so tiny coefficient errors become large prediction errors. Cross-validation on the training data cannot measure this, so the test error is likely somewhat higher than the CV numbers above, particularly for var1.

11. Why these choices

- Degree by cross-validation, not by training error. Training error keeps decreasing with degree; the CV error shows where the model stops generalising.
- Ridge penalty. Needed because the polynomial has far more terms than samples and strongly correlated columns; it makes the fit stable and keeps predictions bounded between sample points.
- Lowest adequate degree. Among tied degrees the smallest was picked, which lowers variance and reduces the risk when extrapolating towards the borders where the test inputs lie.
- No test-set tuning. Test inputs were used only to produce predictions.
- Limitations: one selection protocol (5-fold CV) was used, so the choice among near-tied degrees has some noise; a neighbouring degree could perform equally well. The same penalty is applied to all terms (no per-degree weighting), a simple choice that worked well here.

12. Reproducibility and deliverables

GitHub repository (all training and inference code): <https://github.com/Ayush1patel/ML-Assignment1-Polynomial-Regression>

Path in repository	Content
<code>codes/cv_explore.py, cv_refine.py</code>	degree / alpha search by CV (coarse and fine)
<code>codes/cv_grid.py</code>	full CV grid used for the figures in this report
<code>codes/train_predict.py</code>	final fit on all training rows + writes the predictions
<code>codes/make_report.py</code>	builds this PDF
<code>predictions/BT2024171_pred_var1.csv</code>	1000 predicted y values for var1 test (column y)
<code>predictions/BT2024171_pred_var2.csv</code>	1000 predicted y values for var2 test (column y)
<code>data/, report/, docs/</code>	provided datasets, this report + metrics, assignment text

Run: `pip install -r requirements.txt` then `python codes/train_predict.py` (all scripts take their paths from the repository root, so they can be run from any directory). Fixed random seeds are used for cross-validation, so the numbers in this report reproduce exactly. The prediction files follow the sample submission format: a single column named y, one row per test row, in the same order as the test file.